

第8章: CRUD実装（編集・削除・検索）

この章で CRUD を完成させます。残りの U（Update＝編集）と D（Delete＝削除）を実装し、さらに使い勝手を上げる 検索機能 も追加します。第4章で学んだ動的ルート `[id].tsx`、第6章で作った `fetchBookById / updateBook / deleteBook` 関数を使います。

1. この章のゴール

- 一覧のカードをタップすると編集画面へ移動する
- 編集画面で既存データを初期表示し、変更して更新（Update）できる
- 編集画面から削除（Delete）できる（確認ダイアログ付き）
- 一覧画面に検索ボックスを追加し、タイトル・著者で絞り込む

2. 一覧カードをタップで編集画面へ

まず、第7章の `app/index.tsx` の一覧カードを「押せる」ようにし、押したら `/books/その本のid` へ移動するようにします。`renderItem` の `<View style={styles.card}>` を `<Pressable>` に変えます。

```
// app/index.tsx の renderItem を次のように変更する（変更点のみ抜粋）

renderItem={({ item }) => (
  // View → Pressable に変更。押すと編集画面へ移動する
  <Pressable
    style={styles.card}
    onPress={() => router.push(`/books/${item.id}`)}
    // `/books/${item.id}` : テンプレートリテラル。${item.id}にその本のidが埋め込まれる
    // 例: idが "abc" なら "/books/abc" というパスになり、books/[id].tsx が開く
  >
    <Text style={styles.title}>{item.title}</Text>
    <Text style={styles.author}>著者: {item.author}</Text>
    <View style={[styles.badge, getStatusStyle(item.status)]>
      <Text style={styles.badgeText}>{item.status}</Text>
    </View>
  </Pressable>
)}
```

テンプレートリテラル ``...${ }...``（復習）：バッククォート ``` で囲んだ文字列の中では、`${ }` を使って変数を埋め込みます。``/books/${item.id}`` は「`/books/` に続けて、その本のidをつなげたパス」を作ります。ダブルクォートでの文字列連結（`"/books/" + item.id`）と同じ結果ですが、こちらの方が読みやすいです。

Pressable を **import** しているか確認: 第7章で既に `import { ... Pressable ... } from "react-native";` に含まれています。もし無ければ追加してください。

3. 編集画面を作る（Update）

第6章で作った動的ルート `app/books/[id].tsx` を本実装します。この画面は「①idを受け取る → ②そのidの本をSupabaseから取得して初期表示 → ③変更して更新」という流れです。

▼ このコードがやること（先に日本語で）：「一覧で押した本」を編集・削除できる画面を1枚作ります。まず `useLocalSearchParams` で「どの本を開いたか（id）」を受け取り、`useEffect` で画面表示時にその本のデータを取得して入力欄に初期表示します。あとは更新ボタンで `updateBook`、削除ボタンで（確認ダイアログ付きの）`deleteBook` を呼ぶだけです。これ1枚に CRUD の U（更新）と D（削除）が詰まっています。各行の細かい意味はコード内のコメントで説明します。

```
// app/books/[id].tsx - 編集画面 (Update & Delete)

import { useState, useEffect } from "react";
import { View, Text, TextInput, Pressable, StyleSheet, ScrollView, Alert,
ActivityIndicator } from "react-native";
import { useLocalSearchParams, useRouter } from "expo-router";
import { fetchBookById, updateBook, deleteBook } from "../../lib/books"; // 第6章の関数
(パスは ../../)

const STATUS_OPTIONS = ["未読", "読書中", "読了"];

export default function EditBookScreen() {
  const router = useRouter();
  // useLocalSearchParams : 現在のルートのパラメータを取得するフック
  // books/[id].tsx の [id] 部分を id という名前で受け取れる
  const { id } = useLocalSearchParams<{ id: string }>();

  // 入力欄のstate (第7章の登録フォームと同じ構成)
  const [title, setTitle] = useState("");
  const [author, setAuthor] = useState("");
  const [status, setStatus] = useState("未読");
  const [memo, setMemo] = useState("");
  const [loading, setLoading] = useState(true); // 初期データ読み込み中か
  const [saving, setSaving] = useState(false); // 更新処理中か

  // 画面表示時に、idの本を取得してフォームの初期値にセットする
  useEffect(() => {
    const load = async () => {
      try {
        const book = await fetchBookById(id); // idで1冊取得 (第6章)
        // 取得した値を各stateにセット → フォームに初期表示される
        setTitle(book.title);
        setAuthor(book.author);
        setStatus(book.status);
        setMemo(book.memo ?? ""); // memoがnullなら空文字に(?? で代替)
      } catch (e) {
        Alert.alert("読み込みエラー", "本の情報を取得できませんでした");
        console.log(e);
      } finally {
        setLoading(false);
      }
    };
    load();
  }, [id]); // 依存配列に id。idが変わったら取得し直す

  // 更新ボタンの処理
  const handleUpdate = async () => {
    if (title.trim() === "" || author.trim() === "") {
      Alert.alert("入力エラー", "タイトルと著者は必須です");
    }
  };
}
```

```

    return;
  }
  try {
    setSaving(true);
    // updateBook(id, 新しい内容) : 第6章の更新関数。idの本を上書きする
    await updateBook(id, {
      title: title.trim(),
      author: author.trim(),
      status: status,
      memo: memo.trim() === "" ? null : memo.trim(),
    });
    router.back(); // 一覧へ戻る（戻り先で再取得され反映）
  } catch (e) {
    Alert.alert("更新に失敗しました", "通信状況を確認してください");
    console.log(e);
  } finally {
    setSaving(false);
  }
};

// 削除ボタンの処理（確認ダイアログを出す）
const handleDelete = () => {
  // Alert.alert(タイトル, 本文, ボタン配列) : ボタンを複数置けるダイアログ
  Alert.alert(
    "削除の確認",
    "この本を削除しますか？この操作は取り消せません。",
    [
      { text: "キャンセル", style: "cancel" }, // style:"cancel" : キャンセル用のボタン
      (何もしない)
      {
        text: "削除",
        style: "destructive", // style:"destructive" : 赤字の警告ボ
        タン (iOS)
        onPress: async () => { // 「削除」を押したときだけ実行
          try {
            await deleteBook(id); // 第6章の削除関数
            router.back(); // 一覧へ戻る
          } catch (e) {
            Alert.alert("削除に失敗しました", "通信状況を確認してください");
            console.log(e);
          }
        },
      },
    ],
  );
};

// 初期データ読み込み中はぐるぐるを表示
if (loading) {
  return (

```

```

        <View style={styles.center}>
            <ActivityIndicator size="large" color="#1e40af" />
        </View>
    );
}

return (
    <ScrollView style={styles.container} contentContainerStyle={{ padding: 20, gap: 18 }}>
        {/* タイトル */}
        <View>
            <Text style={styles.label}>タイトル *</Text>
            <TextInput style={styles.input} value={title} onChangeText={setTitle}
placeholder="タイトル" />
        </View>
        {/* 著者 */}
        <View>
            <Text style={styles.label}>著者 *</Text>
            <TextInput style={styles.input} value={author} onChangeText={setAuthor}
placeholder="著者" />
        </View>
        {/* ステータス */}
        <View>
            <Text style={styles.label}>ステータス</Text>
            <View style={styles.statusRow}>
                {STATUS_OPTIONS.map((option) => (
                    <Pressable
                        key={option}
                        style={[styles.statusButton, status === option &&
styles.statusButtonActive]}
                        onPress={() => setStatus(option)}
                    >
                        <Text style={[styles.statusText, status === option &&
styles.statusTextActive]}>{option}</Text>
                    </Pressable>
                ))}
            </View>
        </View>
        {/* メモ */}
        <View>
            <Text style={styles.label}>メモ</Text>
            <TextInput
                style={[styles.input, styles.textarea]}
                value={memo} onChangeText={setMemo}
                placeholder="感想や覚え書き" multiline numberOfLines={4}
            />
        </View>

        {/* 更新ボタン */}
        <Pressable

```

```

        style={ [styles.saveButton, saving && styles.saveButtonDisabled] }
        onPress={handleUpdate} disabled={saving}
      >
        <Text style={styles.saveButtonText}>{saving ? "更新中..." : "更新する"}</Text>
      </Pressable>

      { /* 削除ボタン (赤) */ }
      <Pressable style={styles.deleteButton} onPress={handleDelete}>
        <Text style={styles.deleteButtonText}>この本を削除</Text>
      </Pressable>
    </ScrollView>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f8fafc" },
  center: { flex: 1, justifyContent: "center", alignItems: "center" },
  label: { fontSize: 13, fontWeight: "600", color: "#334155", marginBottom: 6 },
  input: {
    backgroundColor: "#fff", borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8,
    paddingHorizontal: 12, paddingVertical: 10, fontSize: 14,
  },
  textarea: { height: 100, textAlignVertical: "top" },
  statusRow: { flexDirection: "row", gap: 8 },
  statusButton: {
    flex: 1, alignItems: "center", paddingVertical: 10,
    borderWidth: 1, borderColor: "#e2e8f0", borderRadius: 8, backgroundColor: "#fff",
  },
  statusButtonActive: { borderColor: "#1e40af", backgroundColor: "#eff6ff" },
  statusText: { fontSize: 13, color: "#475569" },
  statusTextActive: { color: "#1e40af", fontWeight: "700" },
  saveButton: { backgroundColor: "#1e40af", paddingVertical: 14, borderRadius: 10,
    alignItems: "center", marginTop: 8 },
  saveButtonDisabled: { backgroundColor: "#94a3b8" },
  saveButtonText: { color: "#fff", fontWeight: "bold", fontSize: 15 },
  deleteButton: { paddingVertical: 14, borderRadius: 10, alignItems: "center",
    borderWidth: 1, borderColor: "#fecaca", backgroundColor: "#fef2f2" },
  deleteButtonText: { color: "#dc2626", fontWeight: "bold", fontSize: 14 },
});

```

3.1 重要ポイントの解説

① `useLocalSearchParams` でidを受け取る 動的ルート `books/[id].tsx` の `[id]` 部分の値（どの本を開いたか）を、`useLocalSearchParams` で取得します。 `const { id } = useLocalSearchParams<{ id: string }>()` で、`/books/abc` を開いたなら `id` に `"abc"` が入ります。

`useLocalSearchParams<{ id: string }>()` の山カッコ: `<{ id: string }>` は「受け取るパラメータの型」をTypeScriptに教えています。`books/[id].tsx` なので `id` という文字列が来る、という宣言です。これで `id` を安全に使えます。

② 既存データを初期表示する流れ `useEffect`（画面表示時に1回）で `fetchBookById(id)` を呼び、取得した本の各値を `setTitle` などでもstateに入れます。stateが入力欄の `value` に結びついているので、フォームに既存の値が最初から表示されます。これが「編集」の体験を作る肝です。

③ 削除の確認ダイアログ 削除は取り消せない操作なので、いきなり消さず `Alert.alert` の第3引数にボタン配列を渡して確認します。「キャンセル」と「削除」の2つを置き、「削除」を押したときだけ実際に `deleteBook` を実行します。
`style: "destructive"` は赤い警告色のボタン（iOS）になります。

なぜ確認を挟む？「うっかり削除」を防ぐためです。データを失う・お金が動く・外部に送信する、といった取り返しのつかない操作の前には、必ずユーザーに確認を取るのが良いアプリ設計です。

4. 検索機能を追加する

最後に、一覧画面に検索ボックスを付け、タイトル・著者で絞り込めるようにします。`app/index.tsx` を改良します。検索は「取得済みのデータをアプリ側で絞り込む」方式にします（シンプルで高速）。

▼ このコードがやること（先に日本語で）：一覧画面に検索ボックスを足し、入力した言葉でタイトル・著者を絞り込んで表示します。仕組みはシンプルで、検索キーワードを `keyword` という state に入れ、`useMemo` の中で `filter` を使って「キーワードを含む本だけ」を残した配列を作り、それを `FlatList` に渡すだけです。サーバーに問い合わせ直すのではなく、すでに取得済みの一覧をアプリ側で絞り込むのがポイントです。`useMemo` や `includes` の意味はコード内のコメントとこの後の解説で説明します。

```
// app/index.tsx に検索機能を追加する (主な変更点)

import { useState, useCallback, useMemo } from "react"; // useMemo を追加
import { TextInput } from "react-native"; // TextInput を追加
// (他のimportは第7章のまま)

export default function HomeScreen() {
  const router = useRouter();
  const [books, setBooks] = useState<Book[]>([]);
  const [loading, setLoading] = useState(true);
  const [keyword, setKeyword] = useState(""); // ★追加: 検索キーワードのstate

  // (load関数・useFocusEffectは第7章のまま)

  // ★追加: キーワードで絞り込んだ結果を計算する
  // useMemo : 計算結果を覚えておき、必要なときだけ再計算するフック (無駄な計算を減らす)
  const filteredBooks = useMemo(() => {
    if (keyword.trim() === "") return books; // 未入力なら全件をそのまま返す
    const lower = keyword.toLowerCase(); // 大文字小文字を区別しないため小文字化
    // filter : 条件に合う本だけ残す (第2章)。タイトルか著者にキーワードを含むものを抽出
    return books.filter(
      (b) =>
        b.title.toLowerCase().includes(lower) || // includes : 文字列に含まれるか判定 /
        || は「または」
        b.author.toLowerCase().includes(lower)
    );
  }, [books, keyword]); // booksかkeywordが変わったときだけ再計算

  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#1e40af" />
      </View>
    );
  }

  return (
    <View style={styles.container}>
      {/* ★追加: 検索ボックス */}
      <View style={styles.searchWrap}>
        <TextInput
          style={styles.search}
          placeholder="🔍 タイトルや著者で検索..."
          value={keyword}
          onChangeText={setKeyword}
        />
      </View>

      <FlatList

```



```

        data={filteredBooks} // ★変更: books → filteredBooks (絞り込み後を表示)
        keyExtractor={({item}) => item.id}
        contentContainerStyle={{ padding: 16, gap: 10 }}
        ListEmptyComponent={
            // 検索中かどうかでメッセージを出し分ける (三項演算子)
            <Text style={styles.empty}>
                {keyword.trim() === "" ? "まだ本が登録されていません。" : "該当する本が見つかりません。"}
            </Text>
        }
        renderItem={({ item }) => (
            <Pressable style={styles.card} onPress={() => router.push(`/books/${item.id}`)}>
                <Text style={styles.title}>{item.title}</Text>
                <Text style={styles.author}>著者: {item.author}</Text>
                <View style={[styles.badge, getStatusStyle(item.status)]>
                    <Text style={styles.badgeText}>{item.status}</Text>
                </View>
            </Pressable>
        )}
    />

    <Pressable style={styles.fab} onPress={() => router.push("/new")}>
        <Text style={styles.fabText}>+</Text>
    </Pressable>
</View>
);
}

// styles に検索ボックス用を追加
// searchWrap: { padding: 16, paddingBottom: 0 },
// search: { backgroundColor: "#fff", borderWidth: 1, borderColor: "#e2e8f0",
borderRadius: 8, paddingHorizontal: 14, paddingVertical: 10, fontSize: 14 },

```

useMemo とは？ `useMemo` (ユーズ・メモ) は「重い計算の結果を覚えておき、関係する値が変わったときだけ再計算する」フックです。ここでは「キーワードでの絞り込み」を、`books` か `keyword` が変わったときだけ実行します。本が少ないうちは無くても動きますが、データが増えても軽快に保つための良い習慣です。

includes と toLowerCase :- 文字列.`includes("探す文字")` ... その文字が含まれていれば `true` 。 - 文字列.`toLowerCase()` ... すべて小文字に変換。「ABC」で検索しても「abc」がヒットするよう、両方を小文字に揃えて比較しています。

サーバー側で検索する方法 (発展) : 本書はアプリ側で絞り込みましたが、データが非常に多い場合は Supabase 側で `.ilike("title", \ ${keyword}%)` のように検索させる方法もあります (`ilike`` は大文字小文字を無視した部分一致) 。まずはアプリ側の方式で十分です。

5. 動作確認 — CRUD完成！

`npx expo start` で起動し、すべての機能を通して確認しましょう。

操作	期待する動作
一覧のカードをタップ	編集画面が開き、その本の情報が初期表示される
内容を変えて「更新する」	一覧に戻り、変更が反映されている
「この本を削除」→「削除」	確認後に削除され、一覧から消える
「削除」確認で「キャンセル」	何も起きない（消えない）
検索ボックスに文字入力	タイトル・著者で絞り込まれる
該当なしの語で検索	「該当する本が見つかりません」と表示

これで **CRUD（作成・読取・更新・削除） + 検索** がすべて揃いました。アプリとして「使える」状態になっています！ 🎉

Supabase側でも確認: Supabaseダッシュボードの「Table Editor」で `books` テーブルを開くと、アプリでの追加・更新・削除がデータベースに反映されているのが分かります。アプリとデータベースが本当につながっている実感が得られます。

6. この章のまとめ

- 一覧カードを `Pressable` にし、`router.push(`/books/${item.id}`)` で編集画面へ遷移
- `useLocalSearchParams` で動的ルートの `id` を受け取る
- 編集画面では `useEffect` + `fetchBookById` で既存データを初期表示し、`updateBook` で更新 (Update)
- `Alert.alert` の確認ダイアログ付きで `deleteBook` を実行 (Delete)
- 検索機能を `useMemo` + `filter` + `includes` で実装
- これで **CRUDが完成！**

次の章へ: 機能は完成しました。第9章では、`NativeWind` を導入して見た目を効率よく整え、アプリの完成度をさらに高めます。